

ADR-008 — Serving while indexing, and the one answer that must never be `200`

- **Status:** Proposed. The implementation decision is Jay's; this document exists to make it answerable.
- **Date:** 2026-11-01
- **Deciders:** Jay
- **Supersedes:** none. Fixes a defect in the start-up path chosen by ADR-002 ("the catalog is an in-memory index, rebuilt at start"), and hardens increment 1 of ADR-004.

Context

The `0c6fa84` deploy took the live server **hard down for 10m47s**. Not slow — `/healthz` was *connection-refused* for the whole window, because there was nothing listening to refuse it politely.

What the start-up path actually does

`cmd/yarg-song-server/main.go`, `run()`:

1. line 311 — `ix, err := library.Build(opt.Songs)`, synchronous, walks the entire library
2. ~150 lines of cheap setup (pack cache, features, logging)
3. line 465 — `srv.ListenAndServe()`

The socket is not bound until the index is complete. From outside, **"indexing a large library" and "crashed on boot" are the same observation**, which is also why the deploy script's health patience has to be 1800s: a long index cannot be distinguished from a hang by anything a deployer can see.

`library.Build` is one single-threaded `filepath.WalkDir` (`internal/scan/library.go:38`), and per song `internal/scan/scan.go:138` computes `packageHash` — SHA-256 over the sorted `<lowercased name>:<sha256>` lines, which means **it reads every byte of every file in every song, audio included**, on every restart. The cost is proportional to the bytes in the library and is paid in full each time the process starts.

The measurements

Songs	Index time	Outage	Per song
23 (v0.1.3, 2026-09-15)	~30 s	~30 s	~1.3 s
1,369	3m39s	3m39s	0.160 s
3,067 (`0c6fa84` , tonight)	10m40.186s	**10m47s**, 21:21:31Z–21:32:18Z	0.209 s

"Roughly linear in song count" is a floor, not a model. Per-song cost rose 30% between the last two rows, so linear extrapolation *understates* the next one. The composition explains it, and the composition is measured (below): the 1,698 songs added tonight are loose folders, and a loose folder costs more to index than a single-file container because every file in it must be opened, read and hashed.

What the indexed set is made of — measured, 2026-11-01

The coordinator's read was that the cost is walking and hashing **loose folders**, not decompressing archives, offering `archives: []` from `/api/v1/library` as the evidence.

The conclusion is right and that evidence does not carry it. `Archives()` (`internal/library/library.go:242`) is computed from `worthUnpacking` , which is `SourceZip || Source7z || SourceRar` **only** — `.sng` and `.yargsong` are excluded *by design*, and the function's own comment records 481 `.yargsong` files in this very library. So `archives: []` is silent about 15.7% of the catalogue.

Counted properly, from `source_path` across all 3,067 entries:

Shape	Count	Share
loose folder	2,586	84.3%
`.yargsong`	481	15.7%
`.zip` / `.7z` / `.rar`	**0**	0%

(The first run of that count was wrong and is worth recording: it classified with `os.path.splitext` , and song folders are full of dots — "Mr. Big - To Be With You", "ii (2x bass pedal expert+)" — so ~190 folders were booked as bogus "extensions" and the loose-folder count came out 2,397. Classify against the known container extensions, not against the tail of a name.)

So: **nothing in the indexed set has to be decompressed** — confirmed, but because `.sng` and `.yargsong` are *stored* (there is no `flate` , `gzip` or `zlib` anywhere in `internal/sng`), not because `archives:` was empty.

That distinction is the one the coordinator predicted would matter, and it does: those 481 entries are **single files with one size and one mtime**, which is a far cheaper and far safer cache key than a loose folder whose hundreds of files can each change independently. A differential index would get 15.7% of this library almost free and would have to work for the other 84.3%.

This is more urgent than when it was raised

`/api/v1/library` still reports `built_at: 2026-11-01T21:21:31.894Z` — the start-up index. Eight further packs and the Rock Band 1 DLC set have been unpacked onto the disk since, and the live index does not contain them. **The next restart therefore indexes more than 3,067 songs**, and at 0.209 s/song and rising it will beat 10m47s.

The crux: an empty library and an unbuilt one look identical

The obvious fix — bind the listener first, swap the index in when it is ready — has the machinery waiting for it. `library.Store.Replace` is the atomic swap, and `api.OnUnpacked` already uses it correctly: it builds a *new* index while the old one keeps serving, then swaps. **The rebuild path is already non-disruptive. Start-up is the one path that blocks.**

But "bind early and serve the index you have" is not a smaller version of the fix. It is a worse bug, and it destroys data on someone else's machine.

`internal/syncclient/syncclient.go:443`, `prune()`:

```
all, _, err := postHave(ctx, opt, base, nil)   // the server's FULL set
...
for h := range local {
    if _, ok := onServer[h]; ok { continue }
    os.Remove(filepath.Join(opt.Dest, h+".sng"))
}
```

A server that answers `200` with an empty list while it is still indexing is telling the truth about its index and a lie about its library. A client running with `--prune` believes it and **deletes the player's entire songs folder**. A ten-minute outage is an inconvenience; this is silent, remote, irreversible data loss, and it would be *caused by the fix*.

The repo already states this principle, in `run()`'s refusal to start on a missing library: "*The server is up and has no songs*" *and* "*the path is wrong*" *look identical from a client.* An unbuilt index served as an empty one is that same ambiguity, handed to the one caller that acts on it destructively.

What rescues the server-only fix: the shipped client already fails closed

`postHave` (`syncclient.go:260`) rejects any non-200:

```
if resp.StatusCode != http.StatusOK {
    return nil, 0, fmt.Errorf("syncclient: POST /api/v1/have returned %s", resp.Status)
}
```

and `Run` calls it **first**, at line 172, before any download and long before `prune`:

```
missing, total, err := postHave(ctx, opt, base, have)
if err != nil {
    return nil, err
}
```

So a **503 aborts the sync immediately, downloads nothing, and never reaches `prune`.**

That is the finding that settles the shape of this change, and it corrects the brief this ADR was written from. The server half and the client half do **not** have to land together — *provided the server answers with a status, not with an empty body*. The already-released client is safe against a 503 today, by construction, because it fails closed. It is unsafe against exactly one thing: a `200` that under-reports the library.

The invariant, stated once: `POST /api/v1/have` must never answer `200` while the index is incomplete. Everything else in this ADR follows from that sentence.

Note what this means for the genuinely-empty case: a server with a real, fully-indexed, empty library *should* answer `200 []`, and a pruning client *should* then delete everything, because that is correct. The distinction between "nothing yet" and "nothing at all" is carried by the **status code**, which is why it cannot be carried by the payload.

Decision

Bind the listener before building the index, and gate every index-dependent answer on readiness — with a status code, never with a partial payload.

1. **Build in the background.** Bind and serve immediately; run `library.Build` in a goroutine and publish with the existing `Store.Replace`. No new concurrency primitive: this is the path `OnUnpacked` already takes.

2. **Index-dependent endpoints return `503` with `Retry-After` until the first index is published.** `POST /api/v1/have`, `GET /api/v1/songs`, `GET /api/v1/songs/{hash}`, `GET /song/{file}`, `GET /api/v1/library`, and the browse page. `/version` and `/api/v1/features` stay available — they do not depend on the index, and during a ten-minute start-up they are exactly what an operator wants to be able to ask.

3. **Publish once, not progressively.** The index becomes visible when it is complete. A progressively-published index makes `/have` answer an incomplete truth, which is the prune hazard again arriving by a different road, and it makes `package_hash` ETags appear and change under clients mid-sync. The gate is binary on purpose.

4. **`/healthz` must distinguish "up, indexing" from "ready", and the deploy gate must assert ready.** Today it is `internal/httpapi/api.go:127` and writes `ok` unconditionally. This is the part that must not be split from item 1: bind the listener earlier without fixing `healthz`, and the deploy script's health loop starts passing the moment the socket opens — on a server that cannot serve a single song. **That converts tonight's visible eleven-minute outage into an invisible one, which is worse than leaving it alone.**

5. **The client change is legibility, not safety, and is therefore separable.** `yarg-sync` against an indexing server today fails with `POST /api/v1/have returned 503 Service Unavailable` — correct, and unreadable. It should say the server is still indexing and, for a long-running sync, wait and retry rather than exit. Worth doing; not a prerequisite for items 1–4.

Which increment this lands in

Increment 1 of ADR-004 — the managed mirror folder — which has already shipped. This is a correctness fix to the increment that is live, not a step toward 2 or 3. It touches no `YARG.Core` seam, needs nothing from upstream, and is not blocked on anybody's answer. Increment 2 is recommended for skipping and increment 3 is a performance

feature; neither is relevant here. The `prune` hazard exists *because* increment 1 put a real sync client in front of this server, and that client is the only caller that can turn a server-side ambiguity into deleted files.

Consequences

- **Hard downtime goes to zero**, regardless of library size, and stays zero as the library grows. That is the whole point: it decouples availability from index cost instead of making index cost smaller.
- **Clients and monitors get a readable answer** during start-up. A 503 with `Retry-After` is something a client, a reverse proxy and a human can each act on correctly; connection-refused and an empty `200` are each unreadable, in opposite directions.
- **The window gets *longer*, not shorter.** Today the server is unreachable for eleven minutes; afterwards it is reachable and refusing song requests for eleven minutes. This is a better failure, not a faster one, and it should not be described as a performance fix.
- **A deploy gate that asserts only "the socket is open" becomes wrong.** Item 4 is load-bearing and `docs/DEPLOY-VAULT2.md` needs updating in the same change.
- **Indexing cost is untouched and still scales with library bytes.** Two candidates, both deliberately out of scope here:
- *Parallelise the walk.* Bounded workers; the array is the likely ceiling, so the win is unknown until measured.
- *Persist the index and skip re-hashing unchanged songs,* keyed on path+size+mtime. This is the real answer to "scales with library size" and it is the dangerous one: `package_hash` is both the `ETag` and the sync identity, so one stale cache entry serves wrong bytes under a hash a client trusts. It needs its own ADR, its own tests and an oracle run. It must not ride along with an availability fix. The composition measured above says it would get the 481 single-file containers nearly free and have to earn the other 2,586.

What this ADR does not establish

- **Where the 10m40s actually goes.** The split between directory walking, chart SHA-1 and whole-file SHA-256 is *not measured*. The code says `packageHash` reads every byte, and that is the obvious suspect, but no profile was taken. A timing run was deliberately not done tonight: archives were being moved onto that box while this was written, so a run would have both contended with it and been contaminated by it. Anything about *which part* dominates is a prediction from the source until someone measures it.
- **That 503 is handled well by every client, as opposed to safely.** `yarg-sync` is established above to fail closed. YARG itself, reading a mirror folder on disk under increment 1, never talks to this server at all. Any third-party caller is unknown.
- **The per-song cost model.** Three points, two of them with different library composition, do not separate "superlinear in count" from "the mix shifted toward loose folders". The consequence is the same either way — linear extrapolation is a floor — but the cause is not settled.